

共享型锁又称为**读锁**(read lock)。若事务 T 对数据对象 A 加上 S 锁, 则事务 T 可以读 A 但不能修改 A , 其他事务只能再对 A 加 S 锁, 而不能加 X 锁, 直到 T 释放 A 上的 S 锁为止。这就保证了其他事务可以读 A , 但在 T 释放 A 上的 S 锁之前不能对 A 做任何修改。

排他型锁与共享型锁的控制方式可以用图 12.4 所示的相容矩阵 (compatibility matrix) 来表示。

T_1	T_2		
	X	S	—
X	N	N	Y
S	N	Y	Y
—	Y	Y	Y

注: Y=Yes, 相容的请求; N=No, 不相容的请求。

图 12.4 封锁类型的相容矩阵

在图 12.4 所示的封锁类型的相容矩阵中, 最左边一列表示事务 T_1 已经获得的数据对象上锁的类型, 其中横线表示没有加锁。最上面一行表示另一事务 T_2 对同一数据对象发出的封锁请求。 T_2 的封锁请求能否被满足用矩阵中的 Y 和 N 表示, 其中 Y 表示事务 T_2 的封锁要求与 T_1 已持有的锁相容, 封锁请求可以满足; N 表示 T_2 的封锁请求与 T_1 已持有的锁冲突, T_2 的请求被拒绝。

12.4 封锁协议

在运用 X 锁和 S 锁这两种基本封锁类型对数据对象加锁时, 还需要约定一些规则。例如, 何时申请 X 锁或 S 锁、持锁时间、何时释放等。这些规则称为**封锁协议**(locking protocol)。对封锁方式制定不同的规则, 就形成了各种不同的封锁协议。本节介绍三级封锁协议。对并发操作的不正确调度可能会带来丢失修改、不可重复读和脏读等数据不一致性问题, 三级封锁协议分别在不同程度上解决了这些问题, 为并发操作的正确调度提供了一定的保证。不同级别的封锁协议达到的数据一致性级别是不同的。

1. 一级封锁协议

一级封锁协议是指事务 T 在修改数据 R 之前必须先对其加 X 锁, 直到事务结束才释放。事务结束包括正常结束 (COMMIT) 和非正常结束 (ROLLBACK)。

一级封锁协议可防止丢失修改, 并保证事务 T 是可恢复的。例如图 12.5(a) 使用一级封锁协议解决了图 12.2(a) 中的丢失修改问题。

图 12.5(a) 中事务 T_1 在读 A 进行修改之前先对 A 加 X 锁, 当 T_2 再请求对 A 加 X 锁时被拒绝, T_2 只能等待 T_1 释放 A 上的锁后获得对 A 的 X 锁, 这时它读到的 A 已经是 T_1 更新过的值 15, 再按此新的 A 值进行运算, 并将结果值 $A=14$ 写回到磁盘。这样就避免了丢失 T_1 的修改。

在一级封锁协议中, 如果仅读数据而不对其进行修改是不需要加锁的, 所以它不能保证可重复读和不脏读。

2. 二级封锁协议

二级封锁协议是指在一级封锁协议基础上, 增加事务 T 在读取数据 R 之前必须先对其加 S 锁, 读完后即可释放 S 锁。

二级封锁协议除防止了丢失修改, 还可进一步防止脏读。例如图 12.5(b) 使用二级封锁协议解决了图 12.2(b) 中的脏读问题。

图 12.5(b) 中, 事务 T_1 在对 C 进行修改之前, 先对 C 加 X 锁, 修改其值后写回磁盘。这时 T_2 请求在 C 上加 S 锁, 因 T_1 已在 C 上加了 X 锁, T_2 只能等待。 T_1 因某种原因被撤销, C 恢复为原值 100, T_1 释放 C 上的 X 锁后 T_2 获得 C 上的 S 锁, 读 $C=100$ 。这就避免了 T_2 脏读。

T_1	T_2	T_1	T_2	T_1	T_2
① XLOCK A		② XLOCK C		① SLOCK A	
② R(A)=16		R(C)=100		SLOCK B	
③	XLOCK A	C=C*2		R(A)=50	
④ A=A-1	等待	W(C)=200		R(B)=100	
W(A)=15	等待	②	SLOCK C	A+B=150	
COMMIT	等待	③ ROLLBACK	等待	②	XLOCK B
UNLOCK A	等待	(C恢复为100)	等待	等待	等待
⑤	获得XLOCK A	UNLOCK C	等待	③ R(A)=50	等待
	R(A)=15	④	获得SLOCK C	R(B)=100	等待
	A=A-1	⑤	R(C)=100	A+B=150	等待
⑥	W(A)=14		COMMIT	COMMIT	等待
	COMMIT		UNLOCK C	UNLOCK A	等待
	UNLOCK A			UNLOCK B	等待
				④	获得XLOCK B
					R(B)=100
				⑤	B=B*2
					W(B)=200
					COMMIT
					UNLOCK B

(a) 没有丢失修改

(b) 不脏读

(c) 可重复读

图 12.5 使用封锁机制解决三种数据不一致性的示例

在二级封锁协议中, 由于读完数据后即可释放 S 锁, 所以它不能保证可重复读。

3. 三级封锁协议

三级封锁协议是指在一级封锁协议的基础上, 增加事务 T 在读取数据 R 之前必须先对其加 S 锁, 直到事务结束才释放。

三级封锁协议除了防止丢失修改和脏读外, 还进一步防止了不可重复读。例如图 12.5(c) 使用三级封锁协议解决了图 12.2(c) 中的不可重复读问题。

图 12.5(c) 中, 事务 T_1 在读 A 、 B 之前, 先对 A 、 B 加 S 锁, 这样其他事务只能再对 A 、 B 加 S 锁, 而不能加 X 锁, 即其他事务只能读 A 、 B , 而不能修改它们。所以当 T_2 为修改 B 而申请对

B 的 X 锁时被拒绝, 只能等待 T_1 释放 B 上的锁。 T_1 为验算再读 A 、 B , 这时读出的 B 仍是 100, 求和结果仍为 150, 即可重复读。 T_1 结束后释放 A 、 B 上的 S 锁, 此时 T_2 才能获得对 B 的 X 锁。

上述三级封锁协议的主要区别在于什么操作需要申请封锁, 以及何时释放锁(即持锁时间)。三级封锁协议可以总结为表 12.2。表中还指出了不同的封锁协议使事务达到的一致性级别是不同的, 封锁协议级别越高, 一致性程度越高。

表 12.2 不同级别的封锁协议和一致性保证

封锁协议	X 锁		S 锁		一致性保证			隔离性 级别保证
	操作结束 释放	事务结束 释放	操作结束 释放	事务结束 释放	不丢失 修改	不脏读	可重 复读	
一级封锁协议		✓			✓			读未提交
二级封锁协议		✓	✓		✓	✓		读已提交
三级封锁协议		✓		✓	✓	✓	✓	可串行化

12.5 活锁和死锁

和操作系统一样, 封锁的方法可能引起活锁(livelock)和死锁(deadlock)等问题。

12.5.1 活锁

如果事务 T_1 封锁了数据 R , 事务 T_2 又请求封锁 R , 于是 T_2 等待; T_3 也请求封锁 R , 当 T_1 释放了 R 上的封锁之后系统首先批准了 T_3 的请求, T_2 仍然等待; 然后 T_4 又请求封锁 R , 当 T_3 释放了 R 上的封锁之后系统又批准了 T_4 的请求…… T_2 有可能永远等待, 这就是活锁的情形, 如图 12.6(a) 所示。

避免活锁的简单方法是采用先来先服务的策略。当多个事务请求封锁同一数据对象时, 封锁子系统按请求封锁的先后次序对事务排队, 数据对象上的锁一旦释放就批准申请队列中第一个事务获得锁。

12.5.2 死锁

如果事务 T_1 封锁了数据 R_1 , T_2 封锁了数据 R_2 , 然后 T_1 又请求封锁 R_2 , 因 T_2 已封锁了 R_2 , 于是 T_1 等待 T_2 释放 R_2 上的锁; 接着 T_2 又申请封锁 R_1 , 因 T_1 已封锁了 R_1 , T_2 也只能等待 T_1 释放 R_1 上的锁。这样就出现了 T_1 在等待 T_2 , 而 T_2 又在等待 T_1 的局面, T_1 和 T_2 两个事务永远不能结束, 形成死锁。如图 12.6(b) 所示。

死锁的问题在操作系统和一般并行处理中已做了深入研究。目前在数据库中解决死锁问题主要有两类方法: 一类方法是采取一定措施来预防死锁的发生; 另一类方法是允许发生死锁, 采用一定手段诊断系统中有无死锁, 若有则进行解除。